基于 Python 模块及 XML 的自动化测试平台设计与实现

罗广恒
（成都天奥技术发展有限公司，四川 成都 610036）

摘　要：针对仪器设备自动测试开发和应用中商用软件体量大、价格贵、学习成本高等问题，利用开源语言 Python 和 XML 配置方式，将自动化测试中共性的用户界面设计、测试用例开发、序列管理、测试执行、数据存储方式等内容设计为统一的应用平台。基于 PySide6 模块的界面设计功能、多线程技术、信号和槽机制，实现了应用平台中各模块的松耦合以及消息的跨对象传递。通过可视化界面配置门限、测试用例、仪表和被测件信息，形成用 XML 语言描述信息和存储数据的序列文件。使用某波导微带线的 S 参数自动测试应用验证本平台软件，结果表明，该平台软件大大降低了用户编写和迁移测试用例的难度，缩短了开发周期，满足小型项目自动测试快速开发和应用的需求。

关键词：自动测试，软件平台，Python 模块
中图分类号：TP311.5　　**文献标识码：**A　　DOI：10.19769/j.zdhy.2024.02.063

Design and Implementation of Automated Testing Platform Based on Python Modules and XML

LUO Guangheng
(Chengdu Spaceon Technology Development Co., Ltd., Chengdu, Sichuan 610036, China)

Abstract: In response to the problems of large volume, high price, and high learning cost of commercial software in the development and application of automatic testing of instruments and equipment, the open source language Python and XML configuration method are used to design a unified application platform for common user interface design, test case development, sequence management, test execution, data storage methods, and other content in automatic testing. Based on the interface design function, multithreading technology, signal and slot mechanism of PySide6 module, loose coupling of various modules in the application platform and cross object transmission of messages have been achieved. Configure thresholds, test cases, instruments, and tested part information through a visual interface to form a sequence file that describes information and stores data in XML language. The S parameter automatic testing application of a certain waveguide microstrip line was used to verify the platform software. The results showed that the platform software greatly reduced the difficulty of users writing and migrating test cases, shortened the development cycle, and met the needs of rapid development and application of automatic testing for small projects.

Key words: automatic test, software platform, python modules

0 引言

仪器设备自动化测试软件主要基于计算机控制仪器设备，利用软件编写测试程序实现测试的自动化。为提高软件的通用性并增强其可扩展能力，一般会将共性的用户界面设计、测试用例开发、序列管理、测试执行、数据存储方式等内容设计为统一平台，软件通过调用不同的测试序列和测试用例文件可实现特定的测试功能[1-4]。其中，TestStand 作为一种商用测试系统集成平台，功能齐备，有良好的扩展性，其包括可定制的测试计划编辑器和具有可自定义的面板视图以及自动测试序列管理工具，支持多种硬件和软件连接，支持多种文档格式、结果报告和变化管理系统等。但作为一种商业软件，其体量大、价格贵，且较复杂，初学者需要花较长的时间去学习和掌握，不利于部分小型项目测试需求的快速开发和应用[5-6]。近年来，Python 语言因其丰富的内置模块和第三方库，可以满足自动测试中的绝大部分应用需求，避免了开发人员重复造轮子现象，在自动化测试软件开发中得到越来越多的应用[7-10]。

本文介绍了一种基于 Python 模块及可扩展标记语言（Extensible Markup Language，XML）的自动化测试平台设计方法，主要利用了 Python 语言的界面设计模块、多线程模块、仪器设备驱动模块、数据库驱动模块等和 XML 灵活的配置方式，开发出了具备通用、移植性好和可扩展能力的自动测试软件，降低了测试用例开发和迁移的难度，提高了仪器设备测试效率。

作者简介：罗广恒，男，1985 年生，工程师，研究方向为无线电计量，E-mail：guangheng_222@163.com。

(C)1994-2024 China Academic Journal Electronic Publishing House. All rights reserved.　http://www.cnki.net

1 Python 模块和 XML

在使用 Python 语言开发自动测试软件时，需要用到许多的内置模块和第三方库。本设计方案中主要应用了界面设计库 PySide6、Visa 驱动模块 PyVisa 以及 Ctypes 模块，同时使用 XML 编写测试序列配置文件。

1.1 PySide6 扩展模块

PySide6 是一种跨平台的开源 GUI 开发框架，与 Qt 库紧密结合，提供了丰富的图形界面控件和 API，支持Python3 编程语言[11]，能够兼容 Qt 的 Designer 工具，使得 UI 的设计与开发更高效。利用 PySide6 的 QtWidgets 模块可以构建 GUI 应用程序所需的基本元素，包括窗口、按钮、标签，以及不同类型的布局等，该模块还包含部分高级控件和对话框，如表格、树形结构、多媒体等；QtCore 提供了 PySide6 所需的核心组件和功能，如事件处理、定时器、文件和目录操作、线程管理、日期和时间处理等；QtGui 提供了 GUI 所需的辅助组件和功能，如图像处理、字体选择、颜色选择等；QtSql 提供了数据库交互的支持，包括 MySQL、SQLite、PostgreSQL 等；QtXml 提供了 XML 文档操作的支持，包括 DOM 和 SAX 2 种解析方式。除这些自动测试软件界面开发常用模块外，PySide6 还包括 QtSvg、QtPrintSupport 等模块，可以作为后续功能扩展应用。

1.2 仪器设备驱动模块

PyVisa 驱动模块是对 NI 公司提供的底层 Visa 接口进行 Python 封装[12]，可以实现对串口、GPIB、VXI、TCP/IP、USBTCM 等接口仪器的支持，提供仪器通信协议支持和数据格式转换，支持发送和接收指令及数据。在 Visa 之上是 PyVisa 的核心库，其是一个用 Python 编写的库，包含了部分 Visa 常用操作，如设备初始化、查询、配置和命令交互。而在 PyVisa 核心库之上，是针对不同硬件/软件接口的具体实现，为 Python 应用程序提供了一种使用 Python 自带的底层 I/O 接口访问 VISA 设备的方式。在创建测量仪器类模型时，可以通过继承 PyVisa 驱动模块的方式，将测量仪器的功能封装成类函数，在函数中发送或接收程控指令及数据来实现特定的测量需求。

虽然 PyVisa 能够支持市面上绝大多数仪器的驱动控制，但对于自动测试系统中的被测装备、开关矩阵和部分没有 Python 封装的接口驱动，需要开发人员自行将厂家提供的驱动文件封装成 Python 文件。此时一般会用到Python 的 Ctypes 模块，其允许 Python 代码调用和使用 C 语言、C++、FORTRAN 等编写的动态链接库（如.so 文件或.dll文件）中的函数和变量。

1.3 XML 配置文件

XML 是一种具有清晰结构的标记语言，容易阅读和理解，能够方便描述数据信息和存储数据[13]。XML 的格式由标签、属性、数据、注释、处理指令组成，在描述测试序列时，可轻松地将门限配置、用例配置、仪表配置、被测件配置等信息分类到不同的标签中进行格式化描述，并保存为文本文件。使用 PySide6 中的 QtXml 模块，可以实现对 XML 配置文件的读取和写入操作，当用户需要进行测试用例迁移或改变测试环境中的某些配置时，只需通过上层应用界面进行简单设置即可修改底层的 XML 文件。使用 XML 作为本设计方案的配置文件，与 JSON 等其他数据描述和存储方式相比更具有优势，如 XML 支持数据、文件、结构和样式等多重语义，其标签化结构和属性值结构更具有可读性，可以方便添加新的标签和属性，具有很强的扩展性和灵活性。

2 系统方案

2.1 软件架构

软件架构是软件系统的顶层结构，明确了软件系统骨架，包括子系统、模块、组件以及它们之间的协作关系、约束规范、指导原则等[14]。分解主要功能模块，确定本方案中自动化测试平台软件由应用界面、用例开发模块、序列管理模块、运行引擎模块、数据采集与处理模块、系统配置模块、底层驱动与插件、本地数据库等构成，各部分有序结合，通过相互调用完成自动测试的不同业务需求。自动化测试平台软件架构框图如图 1 所示。

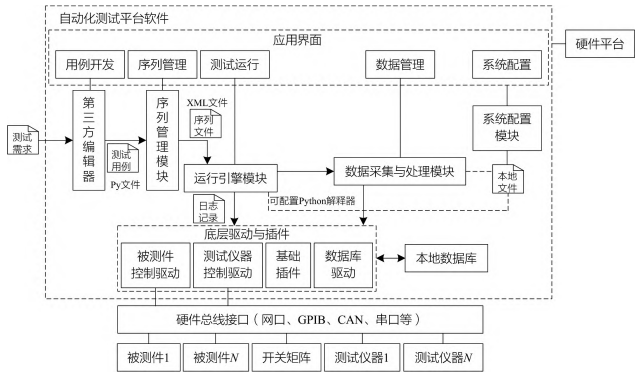

图 1　自动化测试平台软件架构框图

用例开发模块集成第三方编辑器，用于测试用例开发，生成可执行函数的 Python 测试用例文本。序列管理模块对门限、测试用例、仪表、被测件等进行配置，并规定测试用例的执行流程，生成 xml 格式的序列文件。运行引擎模块，以序列文件为入口，通过用户配置的 Python 解释器，加载测试用例，创建被测件对象和测试仪器对象，调用底层驱动控制硬件完成自动测试，并通过数据库驱动保存测试结果到本地数据库，运行过程控制包括开始、暂停及停止操作。数据采集与处理模块通过数据库驱动连接本地数据库，完成测试数据查询、修改、删除等操作，也可根据自定义模板生成测试报告。系统配置模块完成解释器配置、第三方包管理、本地文件路径配置等操作。自动测试平台软件运行在装载 Win7 及以上操作系统的计算机中，要求

(C)1994-2024 China Academic Journal Electronic Publishing House. All rights reserved.　http://www.cnki.net

计算机具备(或能够扩展)测试要求的硬件总线接口,如网口、GPIB、CAN、串口等,用于和被测件、测试仪器、开关矩阵等通信。自动测试相关的配置、测试控制、数据查询、生成报告等操作都通过统一的程序应用界面完成人机交互。

2.2 对象映射技术

在序列文件中,使用 XML 描述门限配置、用例配置、仪表配置、被测件配置信息,在底层的插件与驱动库中,设计对应的 Python 类进行对象映射。在程序初始化过程中,首先读取 XML 中的信息,根据标签属性将 XML 信息映射到测试用例文件中不同的类对象,每个类对象的属性将携带差异化信息,这些类对象在测试用例执行过程中,除具备底层插件与驱动赋予的功能函数外,还将 XML 中的配置信息引入用例执行中,如测试仪器及被测件信息显示、测试用例关联测试仪器初始化、测试结果门限阈值判断等。此做法的好处为:当测试环境发生变化时,通过修改序列文件中的 XML 数据,即可轻松更换对象携带的信息。对象映射框图如图 2 所示。

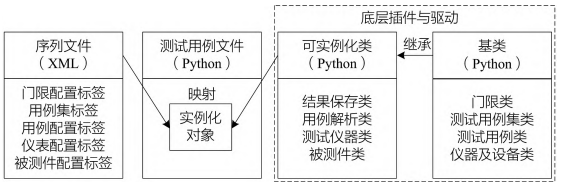

图 2 对象映射框图

2.3 信号和槽机制

PySide6 中信号和槽机制是 Qt 框架一个非常重要的组件,用来实现对象之间的通信,能够有效解决程序模块之间的耦合,使程序更加模块化、灵活、可扩展。信号和槽之间通过名称连接,当槽发生变化时,无需修改信号的代码,这大大提高了程序的可维护性和可扩展性。信号和槽机制能够支持多种参数类型,如整数、字符、字符串、浮点数等,以及自定义类型的参数。信号和槽机制提供了自动化的连接管理,Qt 框架会自动连接信号和槽,并负责断开不再需要的连接,大大简化了程序开发中连接的管理。在本方案中,应用界面和运行引擎模块、数据采集与处理模块之间的通信将运用信号和槽机制,从而实现模块松耦合及模块间信息的及时传递,如测试结果显示和存储、弹窗消息显示等。

2.4 创新点及优势

2.4.1 通用化应用平台

本方案将自动化测试中共性的用户界面设计、测试用例开发、序列管理等内容设计为 1 个统一平台,基于 PySide6 模块的界面设计功能、多线程技术、信号和槽机制,实现了应用平台中各模块的松耦合以及消息的跨对象传递。对测试用例和执行平台进行分离,测试用例的开发者基于底层插件和驱动提供的环境,按一定的规则即可独立开发和发布测试用例,因此,只需将注意力放在测试流程和逻辑的实现上,而无需关心顶层的应用情况。

2.4.2 测试序列管理

测试序列管理实质是对 XML 配置文件的维护,用户通过可视化界面配置门限、测试用例、仪表和被测件信息,修改的内容将保存到 XML 文档中。当测试环境发生变化而测试用例不变时,用户只需通过应用界面更改配置信息,即可快速适应新环境下的测试要求。同时,测试序列中关联了每个测试用例使用的仪器设备,测试用例执行时将只初始化关联的仪器设备,可以方便用户在仪器设备缺失的情况下执行部分测试用例,因此灵活性更高。

2.4.3 移植性及可扩展

由于 Python 语言开源、跨平台和丰富的第三方库特性,大大降低了程序开发的成本和缩短了周期,开发的软件平台可以轻松移植到 Windows、Linux 等操作系统上,当需要进行功能扩展时,只需下载安装对应的 Python 库文件即可,如可扩展 Numpy 库实现对测试数据的分析处理,Scipy 库实现对波形数据的信号处理,Matplotlib 库实现绘图操作等,从而大大增强了自动化测试软件的功能。

3 设计与实现

3.1 测试应用界面

测试应用界面主要基于 PySide6 的 QtWidgets、QtGui 和 QtCore 模块提供的窗口部组件进行开发。首先,使用自带的 Designer 工具绘制基本的图形框架、控件及布局,同时,利用 qss 语言设置样式文件,美化界面,然后将生成的.ui 文件转换成 Python 文件,在 Python 文件中丰富各类初始化函数、功能函数和事件函数等 API 接口,界面间的跳转通过事件响应机制完成。

3.2 序列管理文件

序列管理模块对测试用例进行资源配置、指标门限配置,并规定测试用例和测试用例的执行流程,生成 xml 格式的序列管理文件。序列管理文件中的内容主要包括头部标签、指标门限标签、测试仪器标签、测试序列标签、测试用例标签、被测件标签,各部分以可视化界面方式完成参数配置,具体要求如表 1 所列。

表 1 序列管理文件说明表

标签名称	内容说明	示例
头部标签	包括开发者信息、开发日期、版本信息、其他说明信息等	<head> <author>Simon</author> <date>…</date> <version>v1.0</version> </head>
指标门限标签	包括指标名称、唯一识别名、指标类型、指标范围等	<indicators> <indicator type="float"> <name> S21 测量结果</name> <uniquename>S21</uniquename> <threshold></threshold> </indicator> … </indicators>

(C)1994-2024 China Academic Journal Electronic Publishing House. All rights reserved. http://www.cnki.net

续表

标签名称	内容说明	示例
测试仪器标签	包括本序列中用到的所有仪器 id 和唯一识别码，仪器 id 与数据库中的仪器 id 保持一致	<instruments> <instrument id="123456" uniquename="网络分析仪"/> … </instruments>
测试序列标签	测试序列信息是测试用例的组合，规定了测试用例执行的先后顺序	<procedure> <group name="波导微带 S 参数测量"> <case/> … </group></procedure>
测试用例标签	测试用例信息与脚本文件的测试函数保持名称一致，并关联当前测试用例需要匹配的测试仪器 id 和指标门限等作为子信息	<case> <function args="">初始化 </function> <instrument id="123456"/> </case>
被测件标签	被测件信息包括被测件的 id、名称、唯一识别名、制造商、型号、是否程控、程控地址、备注等信息	<uut id="1"> <name>BD 微带线</name> <model>BD</model> <factory>供应链部</factory> <is_remote>True</is_remote> <remote_addr>11.10.10.32</remote_addr> … </uut>

3.3 测试用例开发

用例开发主要基于 Python 语言进行，开发环境将集成第三方的编辑工具，如 PyCharm。PyCharm 是由 JetBrains打造的一款 Python IDE，具备调试、语法高亮、项目管理、代码跳转、智能提示、自动完成、单元测试等功能[15]，是目前使用较广泛的 Python 集成开发环境。

用户使用本软件平台开发测试用例时，需在一定的框架结构下进行。本软件以文件夹(包)的形式组织脚本文件和静态文件，通过序列文件进行路径关联，从而使整个测试项目目录结构清晰有序。用户可通过快捷方式生成初始开发框架，框架中规定所有测试用例将以独立函数的形式进行体现，便于测试运行引擎的识别和调用。测试序列框架结构如图 3 所示。

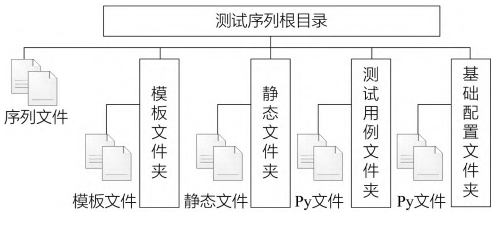

图 3 测试序列框架结构

测试序列文件以 xml 形式组织测试用例、测试仪器和被测设备信息，测试用例名称与测试用例文件的中的函数名称保持一致；测试用例以函数形式封装，测试程序通过面向对象形式调用测试仪器、被测设备和基础操作，对象以“.”调用支持的方法实现相应的功能，中文语句方便用户阅读和二次开发；底层驱动以类的形式进行封装，通过写入、读取和查询等基本操作向测试仪器和被测件发送指令，并回读测试数据，此外，底层驱动还包括数据库驱动和部分基础插件。

3.4 测试运行引擎

测试运行引擎是执行自动测试用例的核心，包括测试过程控制(开始、暂停和停止)，测试信息采集，测试数据处理、异常提醒、日志记录等操作。运行引擎一方面接受用户界面传递的指令，按序列文件的规定执行测试用例，控制硬件的自动执行，另一方面还要将测试过程中产生的数据回传到测试界面或数据库中。运行引擎是决定测试用例和测试界面松耦合的关键，其配置的 Python 解释器以多线程方式独立运行，防止测试用例执行对界面造成阻塞。运行引擎执行完成后的数据与界面交互，采用信号和槽机制传输。当系统崩溃时，测试用例能够恢复到上一次的执行进度。测试运行引擎设计框图如图 4 所示。

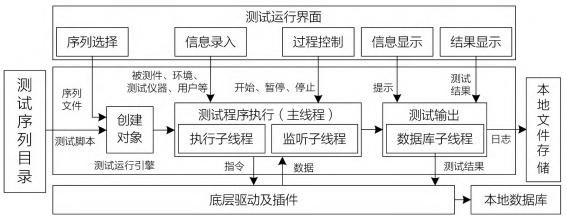

图 4 测试运行引擎设计框图

首先，通过序列文件入口，分析序列文件中的仪器和被测件信息，并动态创建仪器对象、被测件对象及配置参数对象。其次，根据传入用户选择的测试用例，确定关联的仪器对象，并对仪器对象做初步连接，若连接失败，则线程关闭。然后，按先后顺序执行测试用例，用例中涉及提示弹窗等阻塞动作，将等待用户输入后再执行，而结果保存等非阻塞输出信息将以信号和槽机制传递。最后，测试完成，关闭执行线程。测试可以中途暂停和停止，异常情况将以日志文件形式输出。

3.5 数据采集与处理

数据采集主要将不同来源的数据按要求存储或显示到指定位置，如测试数据存储到本地数据库、日志记录存储本地文件、测试数据显示到测试界面等。数据处理主要包括指标合格性判断、数据补偿等，同时也可通过可视化界面查询历史测试数据，并按给定模板生成测试报告。

3.6 底层驱动与插件

底层驱动包括各型测试仪器和被测件的驱动控制文件、数据库驱动文件、基础插件等，主要以 Python 语言进行开发或对其他语言开发的动态库进行封装。开发后的文件既可以独立运行，也可以被测试用例执行调用，便于用户二次开发直接复用，避免重复开发底层驱动。

仪器和被测件的驱动控制文件主要是调用硬件接口封装来创建硬件对象。硬件接口包括 GPIB、串口、网口、CAN 总线等。大部分通用仪器可通过 Visa 驱动实现，并支

(C)1994-2024 China Academic Journal Electronic Publishing House. All rights reserved.　http://www.cnki.net

持SCPI指令,而被测件的驱动文件则较多样化,底层驱动仅开发硬件接口相关的控制命令,上层ICD通信协议将在测试用例设计时考虑。仪器驱动以类的形式封装,通过分析类的继承关系,将类及其父类的属、方法显示在测试用例开发的工具窗口中,方便用户直观调用。

4 应用实例和结果

依据本方案开发的自动化测试软件平台应用在某波导微带线的S参数测量,测量项目包括S21、S11、S22。该波导微带线产品共集成4对端口,测量频率范围为19~32 GHz,每次测量点数为72个,共需测量3次后计算平均值,测试使用是德公司生产的型号为N5244的矢量网络分析仪。使用Python语言引用底层插件和驱动开发测试用例1个,约30行代码,通过函数参数分别对应S21、S11、S22的测量。

使用该自动化测试软件完成单个产品全部测试时间为15 min(含人工换线时间),与人工全手动测试相比,节约约45 min。测试完成后,输出测试结果到本地SQLite数据库中,为方便查看和计算,可通过软件平台功能提取到Excel文件中。测试用例中主要流程代码示例如下:

```
def S参数测量(类型="""S21"):
网络分析仪.连接()
网络分析仪.设置起始频率(19000)
网络分析仪.设置终止频率(32000)
网络分析仪.设置中频带宽(100)
网络分析仪.平均模式("关")
for 序号,频率 in enumerate(频点列表,1):
网络分析仪.设置水平光标(序号,频率)
弹窗.提示("手动进行校准,完毕后点击确定继续!")
网络分析仪.设置类型(类型)
网络分析仪.设置测量格式(格式选项[类型])
值列表=[]
for 序号,频率 in enumerate(频点列表,1):
值列表.append(网络分析仪.读取光标值(序号))
测试结果.存储值(结果=值列表,识别名=类型)
网络分析仪.断开()
```

分析实际应用结果可知,该自动化测试平台软件简化了测试用例编写难度,在测试用例中只涉及对测试流程和逻辑,而相关的界面交互、底层硬件接口控制、指标门限判断等都由平台软件在底层完成。同时,XML配置文件包含了对网络分析仪、指标门限、被测件等信息的描述,当测试环境发生变化时,通过修改XML文件即可轻松应对新的变化。

5 结语

本文针对仪器设备自动测试开发和应用中商用软件体量大、价格贵、学习成本高的问题,利用开源语言Python和XML配置方式设计一种通用、移植性好和可扩展的自动化测试平台,能够满足小型项目的自动测试开发和应用需求。通过分析某波导微带线的S参数自动测量的实际应用效果,该自动化测试软件大大降低了用户编写和迁移测试用例的难度,缩短了开发周期。基于Python语言的开源特性以及强大的内置模块和第三方库,后续可以在测试数据分析、测试数据图形化、网络远程测试等方面进一步扩展平台功能。

参考文献

[1]王红,杨占才,陈洪全,等.民用航空器测试技术体系架构研究[J].测控技术,2022,41(1):28-87.
[2]肖文涛.自动测试系统软件技术的通用性分析[J].通讯世界,2017(24):333-334.
[3]黄娟.基于Python的软件测试自动化平台研究[J].无线互联科技,2021(23):53-54.
[4]李碧涵,胡益诚.机载嵌入式软件的自动化测试架构设计[J].电脑编程技巧与维护,2019(6):13-15.
[5]刘规划.基于LabView和TestStand的并行测试系统设计[J].工业控制计算机,2020,33(4):37-39.
[6]胡典钢.TestStand工业自动化测试管理[M].北京:电子工业出版社,2016.
[7]周阳明,基于Python语言的自动测试系统通用软件平台实现[J].电子设计工程,2019(5):81-85.
[8]余春平.基于Python的EDFA自动测试系统的设计[D].武汉:武汉邮电科学研究院,2020.
[9]杜富强,Python语言下的自动化测试系统设计应用[J].信息与电脑,2019(18):40-42.
[10]冯兴利,洪丹丹,罗军锋,等.基于Selenium+Python的高校统一身份认证自动化验收测试技术研究[J].现代电子技术,2019,42(22):89-97.
[11]李增刚,沈丽.Qt for Python Pyside6 GUI界面开发详解与实例[M].北京:清华大学出版社,2022.
[12]韦彪.基于PyVISA的检测实验室信息管理系统设计初探[J].自动化技术与应用,2019,38(12):57-81.
[13]胡磊,赵羽晗.基于模糊逻辑的模糊XML扩展[J].计算机技术与发展,2020,30(6):1-6.
[14]张英杰,朱雪峰.模式驱动的软件架构设计研究综述[J].计算机科学,2018,45(S2):48-52.
[15]闻雅,高志远,王吉富,等.基于航班数据可视化系统的设计与实现[J].智能计算机与应用,2019,9(3):228-231.

(责任编辑:李 慧)

(C)1994-2024 China Academic Journal Electronic Publishing House. All rights reserved. http://www.cnki.net